תכנון מנגנון משחק: רשת קואורדינטות, פיזור אקראי וחישוב ניקוד

מסמך זה מתאר את ארכיטקטורת החומרה והלוגיקה עבור משחק ה-Gold Miner בסביבת SystemVerilog. המנגנון מחולק לארבעה שלבים עיקריים: חלוקת מסך ה-VGA למערך דו-ממדי, יצירה ופיזור אקראי של אובייקטים (זהב ואבנים), זיהוי התנגשות, וחישוב הניקוד ומהירות התנועה.

1. חלוקת המסך למערך דו-ממדי (Grid)

במקום לשמור את מיקומו המדויק של כל אובייקט ברזולוציה המלאה של המסך (למשל 640x480), אזור החפירה (מתחת לדמות השחקן) מחולק לרשת של משבצות קבועות מראש. גישה זו חוסכת משאבי זיכרון ולוגיקה, ומפשטת משמעותית את זיהוי ההתנגשויות.

- **ממדי הרשת:** חלוקה לוגית (לדוגמה, 4x4 שורות ועמודות).
- **גודל משבצת:** כל תא מייצג גוש של פיקסלים (למשל 64x64 פיקסלים).
- **מיפוי קואורדינטות:** כדי לאתר באיזו משבצת נמצא הפיקסל הנוכחי, אנו מנצלים את העובדה ש-64 הוא חזקה של 2. פעולת ה"חלוקה" מתבצעת פשוט על ידי התעלמות מ-6 הביטים התחתונים (LSB) של קואורדינטות ה-X וה-Y, ושימוש בביטים העליונים (MSB) כאינדקס הגישה למערך.

2. קידוד וייצוג העצמים בזיכרון

כל תא במערך (לדוגמה: MazeBitMapMask) מכיל ערך באורך 3 ביטים, המגדיר בדיוק איזה אובייקט, אם בכלל, נמצא בו.

ערך בביטים	סוג העצם	רדיוס/גודל (פיקסלים)
d'3 0	משבצת ריקה / אובייקט נלכד	-
d'3 1	זהב קטן	16x16
d'3 2	זהב בינוני	32x32
d'3 3	זהב גדול	64x64
d'3 4	סלע קטן	16x16
d'3 5	סלע בינוני	32x32
d'3 6	סלע גדול	64x64

3. אקראיות: יצירה ופיזור בשלב האתחול

תהליך הפיזור מתבצע כאשר המשחק מאופס (למשל כאשר כפתור ה-Reset לחוץ). בזמן זה, המסך אינו מצויר, ומתבצע חישוב מהיר לאכלוס הלוח.

1. **מחולל אקראיות (PRNG):** משתמשים במונה פנימי (Counter) רחב, שרץ בקצב השעון. מכיוון שזמן לחיצת המשתמש על כפתור תחילת המשחק אינו צפוי, ערך המונה באותו רגע נחשב לפסאודו-אקראי.
2. **הגרלת סוג וגודל:** חותכים מקטעים של ביטים מתוך המונה כדי להחליט האם למקם זהב או סלע (ביט 1), ומה יהיה הגודל (2 ביטים).
3. **הגרלת מיקום:** מחלקים את תאי המערך לזוגות או מקטעים, ומשתמשים בביטים מהמונה כדי לקבוע באילו תאים לאכלס את הערכים שנוצרו (ערכים d1'3 עד d6'3), כך שמובטח מספר קבוע של אובייקטים המפוזרים באופן שונה בכל משחק.

4. התנגשות, לכידה, והמרת הנתונים לניקוד

השלב הקריטי ביותר במשחק קורה כאשר הקרס נשלח מטה אל עבר האדמה ופוגע בעצם.

- **זיהוי הפגיעה (Hit Detection):** הפגיעה מזוהה אם הקרס (Hook) מבקש לצייר פיקסל (Drawing Request = 1) באותה קואורדינטה שבה המערך הדו-ממדי שלנו מחזיר ערך ששונה מ-d0'3. מופעל דגל התנגשות (SingleHitPulse) לפריים אחד בלבד כדי למנוע לכידות כפולות.
- **מחיקת העצם מהמסך:** באותו רגע של הפולס, המערכת ניגשת למערך, קוראת את הערך (3 ביטים) ששמור בו למשתנה ביניים, ומיידית דורסת את התא עם הערך d0'3. פעולה זו גורמת לעצם להפסיק להיות משורטט ולהיעלם.
- **שליחת המידע לבקר (Game Controller):** ערך ה-3 ביטים שנשלף מועבר למודול בקר המשחק כדי לפענח אותו.

מימוש בבקר המשחק (Game Controller Logic)

בבקר, הנתון עובר דרך בלוק התניה (Switch/Case) הקובע את מהירות חזרת הקרס ואת ערך הניקוד הממתין. הניקוד מתווסף לקופה הכללית רק כאשר הקרס משלים את החזרה לדמות השחקן.

```
// Pseudo-code implementation within Game Controller
logic [9:0] pending_score;
logic [4:0] pull_speed;

always_ff @(posedge clk or negedge resetN) begin
    if (!resetN) begin
        pending_score <= 0;
        pull_speed <= DEFAULT_SPEED;
    end else if (SingleHitPulse) begin
```

```
        case (captured_object_type) // The 3 bits from the Grid
            3'd1: begin // Small Gold
                pending_score <= 10'd50;
                pull_speed <= FAST_SPEED;
            end
            3'd2: begin // Medium Gold
                pending_score <= 10'd100;
                pull_speed <= NORMAL_SPEED;
            end
            3'd3: begin // Large Gold
                pending_score <= 10'd500;
                pull_speed <= SLOW_SPEED;
            end
            3'd4, 3'd5: begin // Small/Medium Rock
                pending_score <= 10'd0;
                pull_speed <= SLOW_SPEED;
            end
            3'd6: begin // Large Rock
                pending_score <= 10'd0;
                pull_speed <= VERY_SLOW_SPEED; // Penalizes time
heavily
            end
            default: begin
                pending_score <= 10'd0;
                pull_speed <= DEFAULT_SPEED;
            end
        endcase
    end
end
```

כאשר הקרס מזהה שהוא חזר לקואורדינטת ה-Y המקורית שלו (למעלה אצל הכורה), מתבצעת פעולת החיבור: הניקוד הכולל מקבל את ערך ה-pending_score, והרגיסטר הפנימי מתאפס לקראת הזריקה הבאה.